AI Security Issues - Final Project

Adham Yousry Abdulwanis - ID: 23012043
Kirollos Monir Roshdy Sedra - ID: 23012081
Noureldin Ashraf Ahmed - ID: 23012107
Mohamed Elsayed Mohamed - ID: 23010154

1 Overview

A model that achieves high accuracy under normal conditions may be fundamentally fragile when exposed to adversarial conditions (inputs or data designed to mislead it).

In this project we build a neural network classifier for sentiment analysis on the IMDB Movie Reviews dataset, expose it to two distinct adversarial attacks, and apply defense techniques.

The two attacks we implement are:

- Membership Inference Attack: exploiting the model's overfitting to determine whether a given review was part of the training set (IE. a member of the training set), leaking private information about the training data.
- Poisoning Attack: corrupting the training data by flipping 30% of labels before training, causing the model to learn from deliberately wrong signal and degrading its test performance.

2 Dataset

We use the IMDB Movie Reviews Dataset. The dataset contains 50,000 movie reviews, each labeled as either positive or negative sentiment.

The dataset is perfectly balanced, with an equal number of positive and negative reviews. This eliminates class imbalance as a factor when measuring the impact of attacks.

3 Text Preprocessing

Raw text from the dataset contains HTML tags, special characters, numbers, and common words that carry no sentiment signal. So, the following pipeline was applied before vectorization:

- Lowercasing
- HTML tag removal
- Special character removal
- Stopword removal (e.g., 'the', 'is', 'and')
- Lemmatization (e.g., 'running' $\rightarrow$ 'run', 'better' $\rightarrow$ 'good').

Before / After Example:
Before: "One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be
 hooked. They are right, as this is exactly what happened with me..."
After: "reviewer mentioned watching oz episode hooked right exactly happened..."

4 Model

4.1 Vectorization

Text is converted to numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization.

4.2 Model Architecture

We use a Multilayer Perceptron (MLP) implemented in TensorFlow. The architecture is **intentionally** kept simple and without regularization to encourage overfitting (a design choice that sets up the membership inference attack).

Input	(None, 20000)	—
Dense (ReLU)	(None, 256)	5,120,256
Dense (ReLU)	(None, 64)	16,448
Dense (Sigmoid)	(None, 1)	65

Figure 1: model architecture

4.3 Hyperparameters

Optimizer	Adam (lr = 0.001)
Loss function	Binary Crossentropy
Epochs	30
Batch size	256
Validation split	10% of training data
Dropout	None

Figure 2: Hyper-parameters

4.4 Overfitting

The model reaches 100% training accuracy by epoch 5 while validation accuracy stabilizes around 87–88%. This overfitting gap is the intentional vulnerability exploited by the membership inference attack

5 Baseline Evaluation

```
BASELINE PERFORMANCE
  Accuracy : 0.8777
  Precision: 0.8727
  Recall   : 0.8844
  F1-Score : 0.8785

              precision    recall  f1-score   support

    Negative       0.88      0.87      0.88      5000
    Positive       0.87      0.88      0.88      5000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000
```

6 Attack 1 — Membership Inference

6.1 Concept

A membership inference attack attempts to determine whether a specific sample was used to train a model. This is a privacy threat in sensitive domains (medical records,private messages).

The attack exploits a fundamental property of overfitted models: they assign systematically higher confidence to samples they have memorized. By observing only the model's output probability for a given input, an attacker can infer whether that input was a training member or not.

6.2 Method

We use the Adversarial Robustness Toolbox (ART) implementation: (MembershipInferenceBlackBox) with `attack_model_type='rf'`. The attack works as follows:

- A subset of known members (training samples) and non-members (test samples)is used to fit an internal attack classifier.
- The attack classifier takes the target model's output probability as its only inputfeature.
- At inference time, for any new sample, the attack predicts 'member' or 'non-member' based on how confident the target model is.

Attack type	Black-box (output probabilities only)
Internal attack classifier	Random Forest (attack_model_type='rf')
Attack train size	2,000 members + 2,000 non-members
Attack test size	2,000 members + 2,000 non-members

6.3 results

```
MEMBERSHIP INFERENCE ATTACK RESULTS
  Attack Accuracy : 0.6132

  > Baseline (random guessing) : 0.5000
  > Our attack achieves        : 0.6132

              precision    recall  f1-score   support

  Non-member       0.74      0.34      0.47      2000
      Member       0.57      0.88      0.70      2000

    accuracy                           0.61      4000
   macro avg       0.66      0.61      0.58      4000
weighted avg       0.66      0.61      0.58      4000
```

7 Attack 2 — Poisoning Attack

7.1 Concept

A poisoning attack targets the training pipeline rather than the model at inference time. The attacker gains write access to the training data and injects malicious samples before training begins.

Poisoning attacks are realistic in settings where training data is crowdsourced, scraped from user-submitted content, or otherwise collected from sources an attacker can influence.

7.2 Method

The attack focuses on random label flipping, a common adversarial technique designed to introduce "noise" into the decision boundary.

$y_{poisoned} = 1 - y_{original}$

The features X remained untouched; only the ground-truth targets y were manipulated.

Poison rate	30% of training labels flipped
Labels flipped	12,000 out of 40,000
Flip direction	Bidirectional (positive→negative and negative→positive)
Model architecture	Identical to clean model — only labels differ
Selection method	Random uniform sampling (no targeting)

7.3 Results

```
POISONING ATTACK RESULTS
  Metric        Clean Model  Poisoned Model
  ----------------------------------------
  Accuracy           0.8790          0.6852
  Precision          0.8784          0.6793
  Recall             0.8798          0.7016
  F1-Score           0.8791          0.6903

  Accuracy drop: 0.1938 (19.4 percentage points)

Poisoned model — classification report:
              precision    recall  f1-score   support

    Negative       0.69      0.67      0.68      5000
    Positive       0.68      0.70      0.69      5000

    accuracy                           0.69     10000
   macro avg       0.69      0.69      0.69     10000
weighted avg       0.69      0.69      0.69     10000
```

8 Defense Techniques

In this part, we try to fix the problems caused by the attacks. We used two simple defense methods to improve the model and reduce its weaknesses.

8.1 Defense 1 — Data Filtering (Poisoning Defense)

The first defense is used to reduce the effect of the poisoning attack.

We trained a simple Logistic Regression model on the training data and used it to detect suspicious samples. A sample is removed if:

- the predicted label is different from the real label
- the model is not confident enough (confidence ¡ 0.6)

This helps remove noisy or possibly poisoned data before retraining the main model.
Result: We removed 4024 samples from the training data.

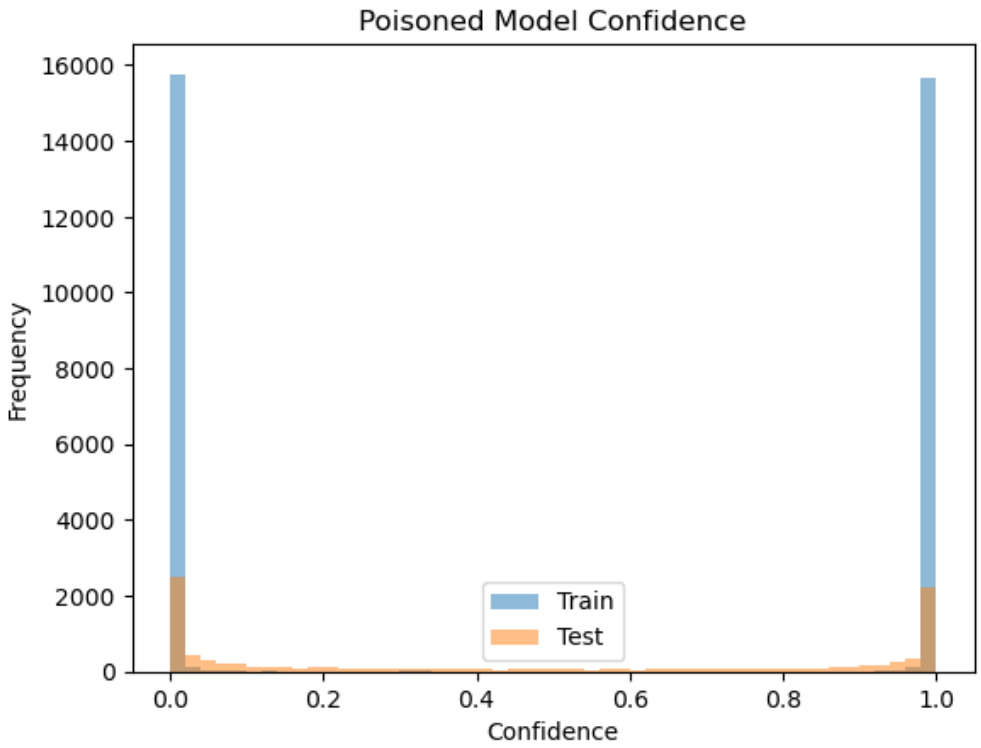

Figure 3: Data filtering effect on training data

8.2 Defense 2 — Probability Clipping + Noise (MIA Defense)

The second defense is used to reduce the membership inference attack.

The idea is simple: the attack works because the model is too confident about its predictions. So we reduce that confidence by:

- clipping probabilities between 0.3 and 0.7
- adding small random noise ($\sigma = 0.05$)

This makes it harder for the attacker to tell if a sample was in the training set or not.

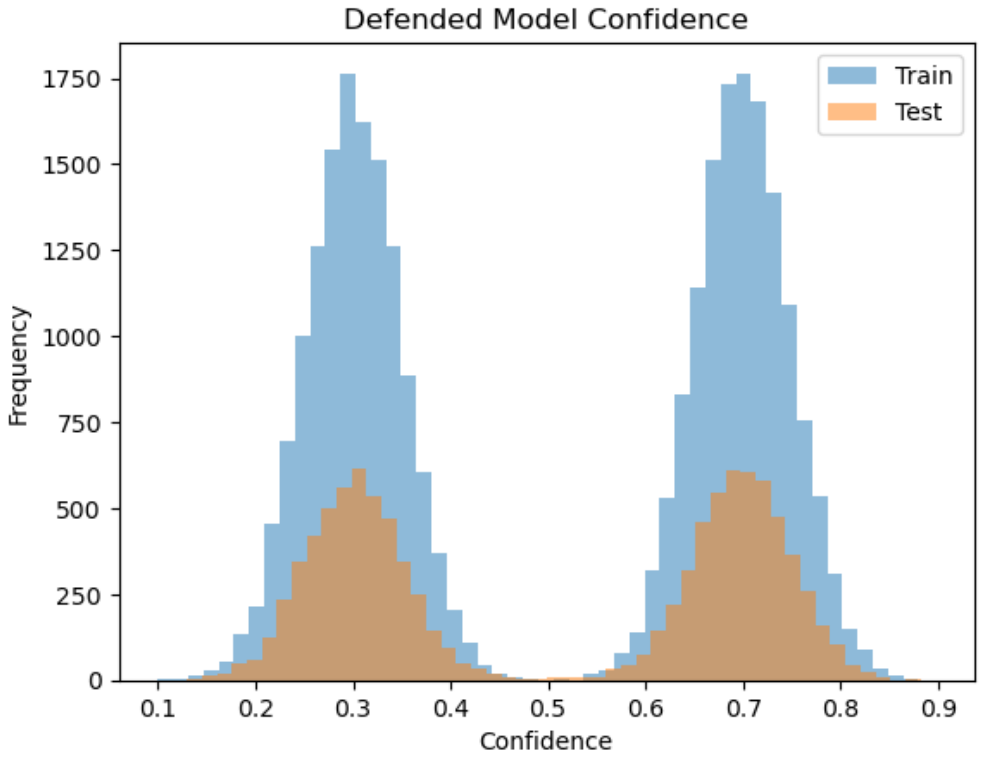

Figure 4: Effect of noise on prediction confidence

8.3 Results — Before vs After Defense

Here is a simple comparison of what changed:

- **Before Defense**
  - Accuracy: 0.8779
  - MIA AUC: 0.7195 (high leakage)
- **After Defense**
  - Accuracy: 0.8880
  - MIA AUC: 0.5023 (almost random guessing)

What this means: The model became more stable after cleaning the data, and at the same time it became much harder for the attacker to detect training samples.

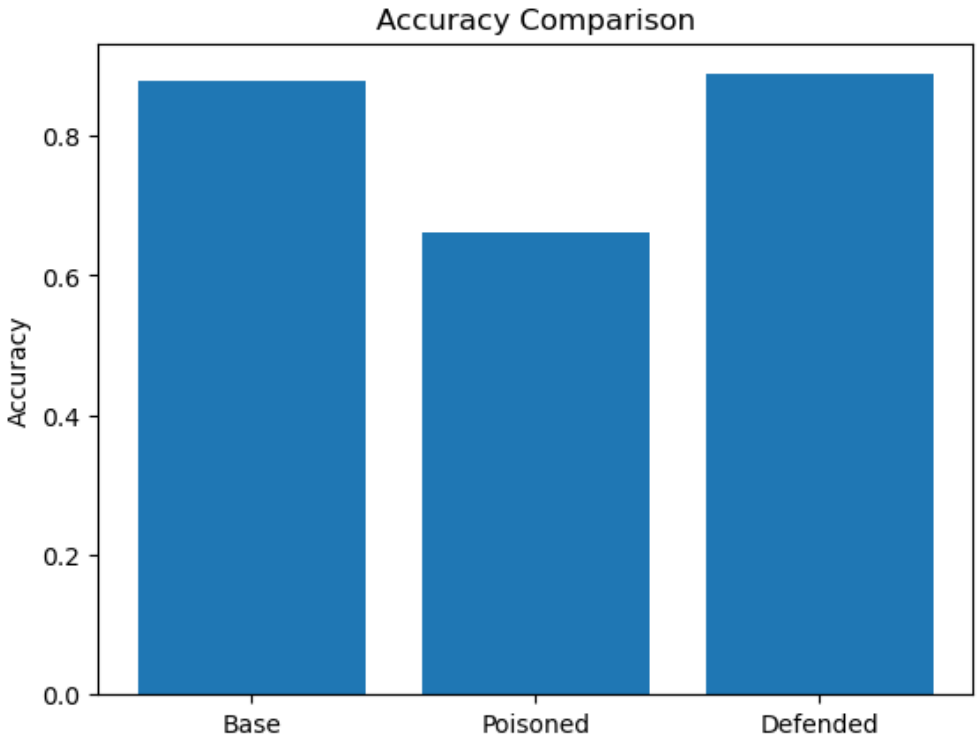

Figure 5: Accuracy before attack, after attack, and after defense

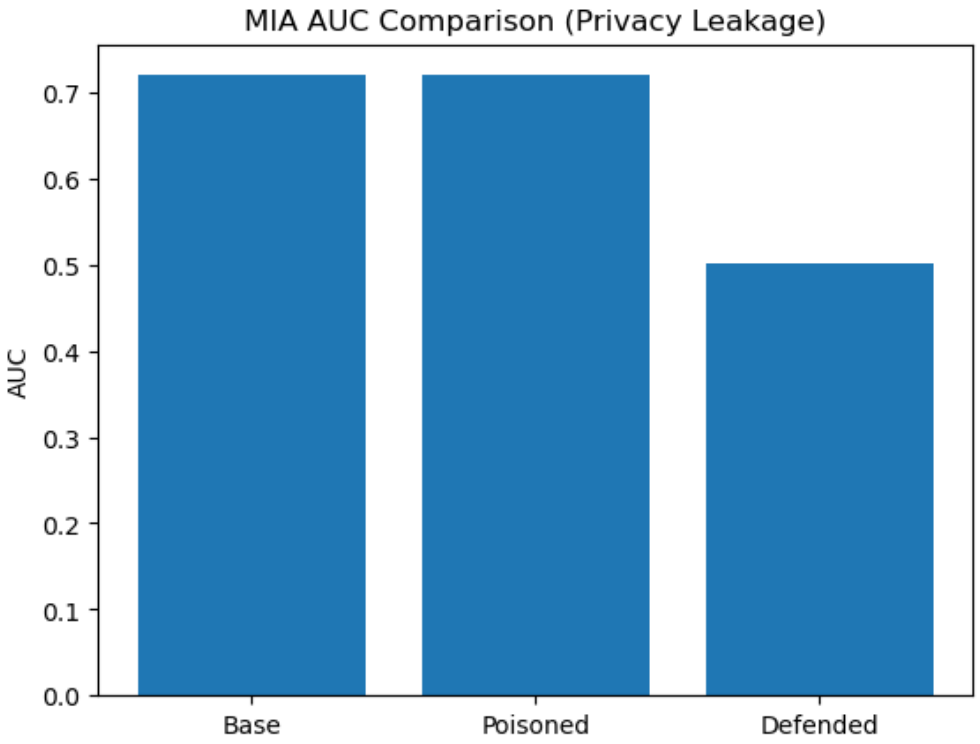

Figure 6: Membership inference attack results (privacy leakage)

9 Conclusion

In this project, we tested how vulnerable a **Machine Learning** model can be to different types of attacks.

We first trained a model for **sentiment analysis on the IMDB dataset**. Even though the model achieved good accuracy, it still had **weaknesses that could be exploited.**

The **membership inference attack** showed that the model was **leaking information** about the training data. The attack score (**AUC = 0.7195**) indicates that the attacker could correctly distinguish training samples from non-training samples better than random chance.

We also tested a **poisoning attack** by flipping **30% of the training labels**. This significantly degraded the model's performance, reducing accuracy from **0.8779 to 0.6624**.

To address these issues, we applied two simple defense techniques: **data filtering** and **adding noise to predictions**. After applying them, the model recovered and slightly improved in performance, reaching **0.8880 accuracy**, while the privacy leakage dropped to nearly random levels (**AUC = 0.5023**).

Overall, the results show that:

- Machine learning models can be easily attacked if they are not properly protected
- Even simple defense techniques can significantly improve robustness
- Privacy leakage is a real and measurable problem, not just a theoretical risk